Experiment No – 6 Date-

PROCESS SYNCHRONISATION

Aim-Implement Algorithm to solve Producer Consumer Problem using Semaphores

Theory –

Process Synchronization:

Process synchronization is a critical concept in operating systems that deals with coordinating the execution of multiple **concurrent processes** to ensure the integrity of shared data and the correct order of operations. The fundamental challenge arises when multiple processes access and manipulate the same data simultaneously, leading to potential **race conditions** and inconsistent results

The Need for Synchronization

When processes run concurrently, their execution order is non-deterministic (it depends on the scheduler). If they share resources (like variables, files, or memory buffers), an unpredictable execution order can lead to errors.

- **Race Condition:** This occurs when two or more processes are accessing and modifying shared data, and the final outcome depends on the exact sequence and timing of their access.
- **Data Inconsistency:** The result of a race condition, where the shared data's state does not reflect the logical outcome of the operations.

The Critical Section Problem

The theoretical problem that synchronization mechanisms aim to solve is the **Critical Section Problem**.

- **Critical Section:** This is the segment of code in a process where **shared resources** (like shared variables or files) are accessed.
- **The Goal:** To design a protocol that ensures that when one process is executing in its critical section, no other process is allowed to execute in its own critical section. This is known as **mutual exclusion**.

Any solution to the Critical Section Problem must satisfy three essential requirements:

1. **Mutual Exclusion:** If process P_i is executing in its critical section, then no other process is allowed to execute in its critical section.
2. **Progress:** If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes not in their remainder section can participate in deciding which will enter its critical section next. This selection cannot be postponed indefinitely.

3. **Bounded Waiting:** There must be a limit on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Synchronization Tools

Operating systems employ various hardware and software tools to implement process synchronization and enforce mutual exclusion:

- **Mutex Locks (Mutual Exclusion Locks):** A simple synchronization primitive that protects a critical section. A process must acquire the lock before entering the critical section and release it upon exiting. Only one process can hold the mutex at a time.
- **Semaphores:** A more general synchronization tool, often described as an integer variable accessed only through two atomic (indivisible) operations:
    - **Wait (or P):** Decrements the semaphore value. If the value becomes negative or zero, the process blocks.
    - **Signal (or V):** Increments the semaphore value, which may unblock a waiting process.
    - **Binary Semaphore:** Functions similarly to a mutex (value is 0 or 1).
    - **Counting Semaphore:** Used to control access to a resource that has multiple instances (value initialized to the number of resources available).
- **Monitors:** A high-level synchronization construct that encapsulates shared data variables and the procedures (methods) that operate on that data within a single structure. Monitors ensure that only one process can be active within the monitor's procedures at any given time, simplifying the implementation of mutual exclusion.

The **Producer-Consumer Problem** is a fundamental synchronization problem in concurrent programming. It describes two asynchronous processes: the **Producer**, which generates data (items), and the **Consumer**, which uses or consumes the data. They share a common, fixed-size data structure called a **buffer** (or queue).

The challenge lies in ensuring that:

1. The **Producer** only adds data when the buffer is **not full**.
2. The **Consumer** only removes data when the buffer is **not empty**.
3. Both processes do **not access the buffer simultaneously** (i.e., mutual exclusion), which could lead to a **race condition** and corrupt the data.

This problem models real-world scenarios like print spooling, inter-process communication (IPC), and handling I/O requests, where tasks run at different speeds and must share resources safely. The solution typically employs **semaphores** to enforce synchronization and mutual exclusion, which is exactly what your provided C code does.

💡 **Key Terminology**

- **Producer:** A process or thread that creates items and places them into the shared buffer.
- **Consumer:** A process or thread that takes items from the shared buffer and consumes them.
- **Buffer (or Bounded Buffer):** A fixed-size, shared memory area where the producer deposits items and the consumer retrieves them. In your code, buffer[BUFFER_SIZE][ITEM_SIZE] is the buffer, modeled as a **circular buffer** using the in and out pointers.
- **Semaphore:** A signaling mechanism (integer variable) used to control access to a common resource in a concurrent environment.
    - **mutex (Mutual Exclusion):** A binary semaphore initialized to **1**. It ensures only one process (producer or consumer) can access the shared buffer at any given time, preventing race conditions.
    - **empty:** A counting semaphore initialized to the **size of the buffer** (BUFFER_SIZE). It counts the number of empty slots. The producer waits on this before adding an item.
    - **full:** A counting semaphore initialized to **0**. It counts the number of occupied slots (items). The consumer waits on this before consuming an item.
- **wait(int *s) (or P operation):** Decrements the semaphore value (*s). If the value becomes negative or zero (in your code, it checks for >0 before decrementing), the process must wait, essentially blocking execution until another process signals it.
- **signal(int *s) (or V operation):** Increments the semaphore value (*s). This typically wakes up a waiting process.
- **Race Condition:** A scenario where the final result of concurrent operations depends on the specific order in which they occur. Semaphores (like mutex) are used to prevent this in the critical section (buffer access).

Example –

BUFFER_SIZE = 5

Initial State: empty = 5, full = 0, in = 0, out = 0.

Initial Buffer: [_, _, _, _, _]

Step	Operation	Item/Action	**Buffer Contents (Slots 0-4)**
0	Initial	-	`[ _, _, _, _, _ ]`
1	Produce	"A1"	`[ **A1**, _, _, _, _ ]`
2	Produce	"B2"	`[ A1, **B2**, _, _, _ ]`
3	Consume	Remove "A1"	`[ _, B2, _, _, _ ]`
4	Produce	"C3"	`[ _, B2, **C3**, _, _ ]`
5	Consume	Remove "B2"	`[ _, _, C3, _, _ ]`
6	Produce	"D4"	`[ _, _, C3, **D4**, _ ]`
7	Produce	"E5"	`[ **E5**, _, C3, D4, _ ]`
8	Consume	Remove "C3"	`[ E5, _, _, D4, _ ]`
9	Produce	"F6"	`[ E5, **F6**, _, D4, _ ]`
10	Consume	Remove "D4"	`[ E5, F6, _, _, _ ]`
11	Produce	"G7"	`[ E5, F6, **G7**, _, _ ]`
12	Consume	Remove "E5"	`[ _, F6, G7, _, _ ]`

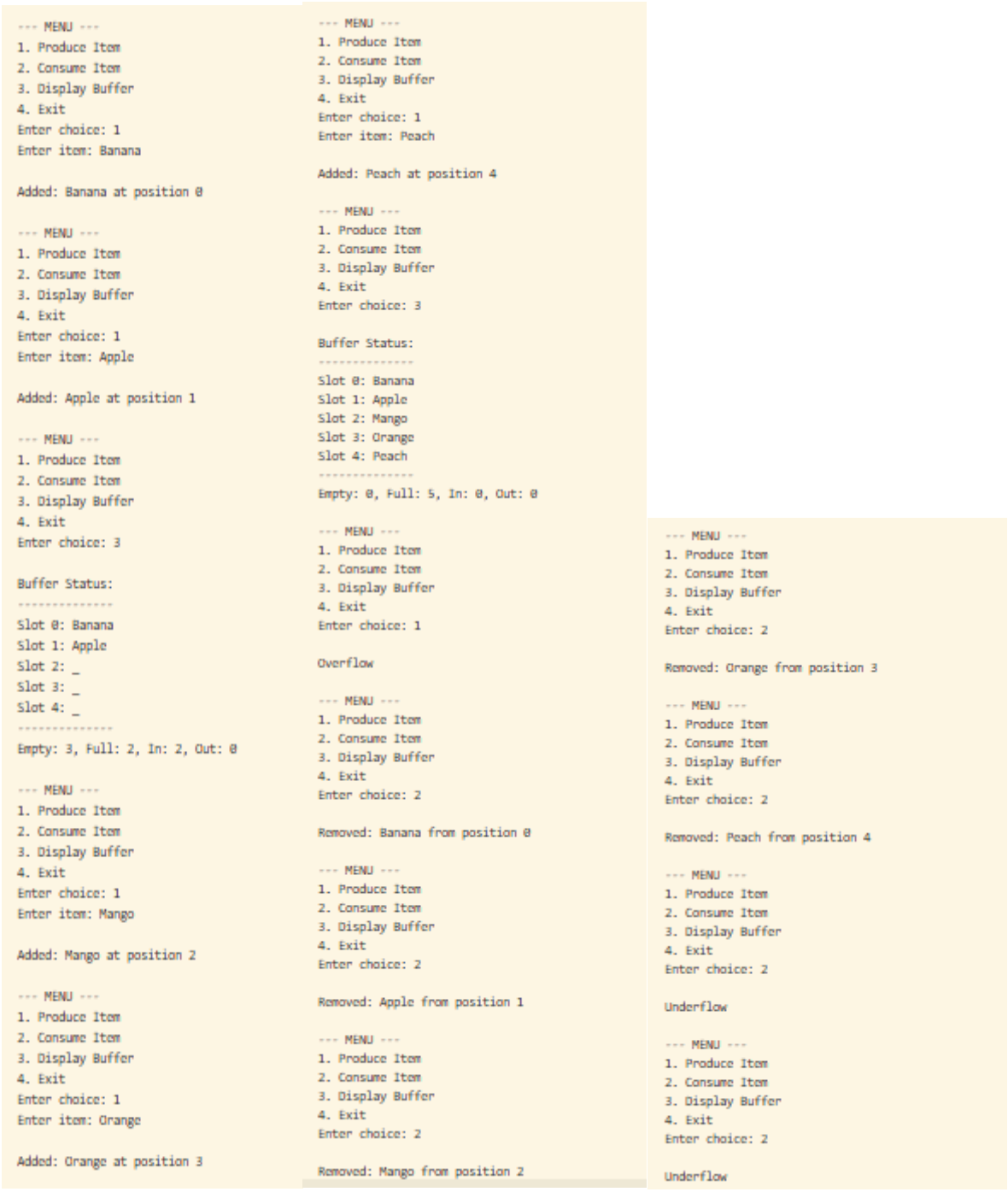
Code –

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define BUFFER_SIZE 5
#define ITEM_SIZE 10
char buffer[BUFFER_SIZE][ITEM_SIZE];
int in = 0, out = 0;
int empty = BUFFER_SIZE;
int full = 0;
int mutex = 1;

void wait(int *s) {
  if (*s > 0)
    (*s)--;
  else
    printf("Wait failed: semaphore value is zero.\n");
}

void signal(int *s) {
  (*s)++;
}

void produce(char item[]) {
  wait(&empty);
  wait(&mutex);
  strcpy(buffer[in], item);
  printf("\nAdded: %s at position %d\n", item, in);
  in = (in + 1) % BUFFER_SIZE;
  signal(&mutex);
  signal(&full);
```

```
}

void consume() {
   wait(&full);
   wait(&mutex);
   printf("\nRemoved: %s from position %d\n", buffer[out], out);
   strcpy(buffer[out], "_");
   out = (out + 1) % BUFFER_SIZE;
   signal(&mutex);
   signal(&empty);
}

void displayBuffer() {
   int i;
   printf("\nBuffer Status:\n");
   printf("--------------\n");
   for (i = 0; i < BUFFER_SIZE; i++) {
      printf("Slot %d: %s\n", i, buffer[i]);
   }   printf("--------------\n");  printf("Empty: %d, Full: %d, In: %d, Out: %d\n", empty, full, in, out);
}
int main() {
   int choice;
   char item[ITEM_SIZE];
   for (int i = 0; i < BUFFER_SIZE; i++)
      strcpy(buffer[i], "_");

   while (1) {
      printf("\n--- MENU ---\n");
      printf("1. Produce Item\n");
      printf("2. Consume Item\n");
      printf("3. Display Buffer\n");
```

```
    printf("4. Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    switch (choice) {
      case 1:
        if (empty == 0)
          printf("\nOverflow\n");
        else {
          printf("Enter item: ", ITEM_SIZE - 1);
          scanf("%s", item);
          produce(item);
        }
        break;
      case 2:
        if (full == 0)
          printf("\nUnderflow\n");
        else
          consume();
        break;
      case 3:
        displayBuffer();
        break;
      case 4:
        printf("\nExiting...\n");
        exit(0);
      default:
        printf("\nInvalid choice\n");      }
  }
  return 0;
}
```

OUTPUT –



```
--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 1
Enter item: Banana

Added: Banana at position 0

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 1
Enter item: Apple

Added: Apple at position 1

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 3

Buffer Status:
--------------
Slot 0: Banana
Slot 1: Apple
Slot 2: _
Slot 3: _
Slot 4: _
--------------
Empty: 3, Full: 2, In: 2, Out: 0

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 1
Enter item: Mango

Added: Mango at position 2

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 1
Enter item: Orange

Added: Orange at position 3

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 1
Enter item: Peach

Added: Peach at position 4

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 3

Buffer Status:
--------------
Slot 0: Banana
Slot 1: Apple
Slot 2: Mango
Slot 3: Orange
Slot 4: Peach
--------------
Empty: 0, Full: 5, In: 0, Out: 0

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 1

Overflow

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 2

Removed: Banana from position 0

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 2

Removed: Apple from position 1

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 2

Removed: Mango from position 2

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 2

Removed: Orange from position 3

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 2

Removed: Peach from position 4

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 2

Underflow

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 2

Underflow
```

Conclusion – Algorithm to solve producer consumer problem was implemented successfully.